

Краткая пояснительная записка к проекту MVP веб-приложения для проведения квизов

Общие сведения

Проект представляет собой MVP веб-приложения для проведения интерактивных квизов в реальном времени по модели, близкой к Kahoot. Система ориентирована на два типа пользователей: организатора и участника. Организатор создаёт квиз, добавляет вопросы, запускает игру и управляет показом вопросов. Участники подключаются к активному квизу по коду комнаты, отвечают на вопросы в реальном времени и видят итоговый лидерборд.

Цель разработки — получить работоспособный прототип, который демонстрирует полный пользовательский сценарий: регистрация, создание квиза, подключение участников, проведение игры, подсчёт баллов и отображение результатов.

Обоснование выбора средств разработки

Для реализации проекта были выбраны современные и практичные инструменты, подходящие для быстрого создания MVP и последующего масштабирования.

Frontend: React + Vite + Tailwind CSS

React был выбран как основной фреймворк для построения интерфейса, поскольку он хорошо подходит для создания динамичных одностраничных приложений и управления состоянием экрана в реальном времени. Vite обеспечивает быстрый старт проекта и удобную сборку. Tailwind CSS позволяет быстро собирать аккуратный адаптивный интерфейс без большого объёма ручного CSS-кода.

Backend: Node.js + Express

Node.js и Express обеспечивают простой и понятный серверный слой на JavaScript, что удобно при использовании единого языка на фронтенде и бэкенде. Это ускоряет разработку, упрощает поддержку и снижает порог входа в проект.

Реальное время: Socket.io

Для квизов критически важна мгновенная синхронизация между организатором и участниками. Socket.io был выбран для передачи событий в реальном времени: старт квиза, переход к следующему вопросу, отправка ответов, обновление лидерборда и список участников.

База данных: Prisma + SQLite

Prisma используется как ORM для удобной и безопасной работы с базой данных. SQLite подходит для MVP, так как не требует отдельного сервера БД и позволяет быстро развернуть рабочий прототип. При необходимости архитектуру можно безболезненно перевести на PostgreSQL или другую промышленную СУБД.

Авторизация: JWT + bcrypt

Для регистрации и входа пользователей используется JWT-аутентификация. Пароли хранятся в хешированном виде с помощью bcrypt, что соответствует базовым требованиям безопасности.

Основные этапы разработки

Этап 1. Проектирование структуры данных

Сначала была разработана модель базы данных, включающая пользователей, квизы, вопросы, варианты ответов и игровые сессии. На этом этапе были определены сущности, связи между ними и основные поля, необходимые для хранения данных о создании и проведении квиза.

Этап 2. Реализация серверной части

Был создан backend на Node.js и Express. Реализованы регистрация и авторизация пользователей, защищённые маршруты для организатора, создание квиза, добавление вопросов и получение списка собственных квизов. Также был подготовлен слой для работы с реальным временем через Socket.io.

Этап 3. Разработка клиентского интерфейса

На фронтенде создано приложение на React с маршрутизацией страниц: главная, вход, кабинет организатора, экран проведения, экран участника, история квизов. Интерфейс адаптирован под сценарии организатора и участника и оформлен с использованием Tailwind CSS.

Этап 4. Настройка WebSocket-обмена

Для обеспечения режима реального времени были реализованы события подключения к комнате, запуска квиза, показа текущего вопроса, отправки ответа и обновления таблицы лидеров. Это позволило участникам видеть процесс синхронно с действиями организатора.

Этап 5. Подсчёт результатов и история участия

Добавлена логика начисления баллов, формирования лидерборда и сохранения результатов игровых сессий. Также реализованы упрощённые страницы истории для организатора и участника, где отображаются проведённые и пройденные квизы.

Этап 6. Тестирование и сборка MVP

После реализации ключевых функций была выполнена проверка работоспособности приложения, исправлены ошибки маршрутизации, взаимодействия сокетов и сборки фронтенда. В результате получен запускаемый MVP-прототип.

Функциональные возможности прототипа

На текущем этапе приложение позволяет:

регистрировать и авторизовывать пользователей;
разделять роли организатора и участника;
создавать квизы и добавлять вопросы;
подключать участников к квизу по коду комнаты;
запускать квиз и переключать вопросы в реальном времени;
принимать ответы участников и начислять баллы;
отображать таблицу лидеров;
сохранять данные об участии и проведённых квизах;
добавлять вопросы разных типов: текстовые и в виде изображения, с одиночным и множественным выбором правильного ответа.

Инструкция по запуску

Требования:

- Node.js 18+ (протестировано на версии 22.15.0)
- npm 10+
- Git

Установка и запуск:

1. Клонировать репозиторий:
`git clone https://github.com/AnnaYurch/quiz-app-vk.git`
`cd quiz-app-vk`
2. Установка backend:
`cd backend`
`npm install`
`npm run prisma generate`
`npm run prisma db push`
3. Создайте файл `.env` в папке backend:
`PORT=3000`
`JWT_SECRET=your_secret_key_here`
4. Запустите backend (в первом терминале):
`npm run dev`
5. Установка frontend (во втором терминале):
`cd frontend`
`npm install`
`npm run dev`
6. Откройте `http://localhost:5173` в браузере

Итог

В ходе разработки был создан работоспособный MVP веб-приложения для проведения квизов в реальном времени. Выбранный стек технологий позволил быстро реализовать полный пользовательский сценарий и обеспечить основу для дальнейшего развития проекта. Архитектура решения остаётся достаточно гибкой для расширения функциональности: добавления загрузки изображений, детальной статистики, расширенных профилей пользователей, полноценных отчётов и последующего переноса на более мощную инфраструктуру.

Приложение

Ссылка на макеты:

<https://www.figma.com/design/JdR8iuFgMry9E2jfVFtliA/%D0%9C%D0%BE%D0%B9-%D0%BB%D0%BE%D0%BA%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9-%D1%81%D0%B0%D0%B9%D1%82?node-id=0-1&t=2GKIv6SoNjfZGUdg-1>

Ссылка на репозиторий: <https://github.com/AnnaYurch/quiz-app-vk/tree/main>